

DRDO GIS

Deployment Geometry System

A Plain-English Guide to Every File and the Workflow

What this system does: You pick a point on a map and give it a size. The app reads the shape of the ground there (from offline terrain files), decides whether the ground is flat or hilly, and draws a suitable deployment footprint - a clean **oval** on flat ground, or a smart **terrain-avoiding blob** on hilly ground. You can then hand-edit the shape, and everything is saved to a database. It runs completely offline as one program that serves both the website and the API.

Technology in one line: a Java (Spring Boot) backend does the map/terrain math and stores data in PostgreSQL+PostGIS; an Angular + OpenLayers website is the screen you interact with. The website is built into the backend, so the whole thing ships as a single runnable file.

Generated as a walkthrough of the repository. Read top-to-bottom, or jump to the file tables in Sections 4 and 5 as a reference.

1. The mental model (read this first)

The whole app is built around **one idea**. If you remember only one sentence, remember this:

Note: You click a spot -> the app reads the **ground heights** around it -> it decides **flat or hilly** -> it draws an **oval (flat)** or a **blob (hilly)** -> it saves the shape and shows it on the map.

Everything in the codebase is just a small helper around that one idea. The files are split up so each one has a single, clear job. Once you know the *job categories*, any file makes sense the moment you see which folder it lives in.

The 3 big pieces (like a restaurant)

Piece	What it is
Frontend	The dining area you see and click - the website (map + buttons). Built with Angular + OpenLayers. Lives in frontend/.
Backend	The kitchen that does the real work - reads terrain, does the math, draws shapes. Built with Java + Spring Boot. Lives in backend/.
Database	The storage room that remembers your saved deployments. PostgreSQL with the PostGIS map add-on.

The frontend (dining area) phones the backend (kitchen). The kitchen cooks and keeps leftovers in the storage room. All of it is packed into one file you just run.

The job categories inside the backend

Java code is organised into layers. A request flows **down** through them and the answer comes back **up**:

Layer	Its job (analogy)
Controller	The doors / reception . Receives a web request and hands it on. Does no real work.
Service	The managers . Run the actual step-by-step workflow and coordinate everything.
Engine	The brain / specialists . Pure math: terrain analysis, shape drawing, coordinate conversion.
Repository	The storeroom clerks . The only ones who read and write the database.
Entity	The shape of a stored row . Describes what a saved thing looks like in the database.
DTO	The envelopes . The exact data shapes sent to and from the website.
Config	The setup crew . Settings and startup wiring.

2. The main workflow - creating a deployment

This is what happens, end to end, when you fill the form and click **COMPUTE GEOMETRY**. Each numbered step names the real file involved.

1

You submit the form (frontend)

`map-view.component` collects your latitude, longitude, frontage, depth, etc. and calls `deployment-api.service`, which sends it to the backend as a web request (HTTP POST to `/api/v1/deployments`).

2

The door receives it (controller)

`DeploymentController` catches the request. First it auto-checks the rules (valid latitude/longitude/size). If they pass, it hands the work to the manager and does nothing else itself.

3

The manager starts the workflow (service)

`DeploymentService.createDeployment()` is the head chef. It saves the basic deployment record and its input parameters, marks the status 'COMPUTING', and runs the next steps in order. The whole method is a single 'transaction': if anything fails, everything is rolled back cleanly.

4

Flat or hilly? (terrain brain)

`TerrainEngine.analyse()` asks `DtedReader` for a grid of ground heights around your point, measures the slope and bumpiness, and decides **isPlanar** (flat) or not. It also produces a 0-100% suitability score.

5

Which way is steep? (directional factors)

If terrain-adaptive is on, `TerrainEngine.directionalSlopeFactors()` checks 8 compass directions and returns a 'shrink amount' for each - small where it's steep.

6

Draw the shape (geometry brain)

`GeometryEngine.generate()` uses the flat/hilly answer: flat -> `BezierEngine.buildEllipse()` (an oval); hilly -> `buildBezierPolygon()` (a smooth blob pulled inward in the steep directions).

7

Save everything (repositories)

The manager turns the shape into map-ready text with `GeoJsonWriter`, then saves the geometry, terrain results, and control points to the database, and flips status to 'READY'.

8

Show it on the map (frontend)

The reply travels back to `map-view.component`, which calls `ol-map.service.renderDeploymentGeometry()` to draw the shape, and `terrain-panel` to show the suitability numbers.

The same flow as a picture

```
deployment-api.service.createDeployment()    <- website sends
|
DeploymentController.create()                <- door receives + checks rules
|
DeploymentService.createDeployment()          <- manager runs all steps:
|  |-- terrainEngine.analyse()                -> flat or hilly?
|      |-- dtedReader.sampleGrid()            -> read ground heights
|  |-- geometryEngine.generate()              -> draw the shape
|      |-- bezierEngine.buildEllipse / buildBezierPolygon
|  |-- geoJsonWriter.toFeatureJson()          -> make it drawable
|  |-- ...Repository.save()                   -> store in database
```

```
|  
ol-map.service.renderDeploymentGeometry()      <- website draws it
```

The other workflows (shorter)

Action	What happens (and the key file)
View / select	Click a name in the left list -> map-view pans the map to it and re-draws its saved shape.
Edit geometry	Click EDIT (only shown for non-oval shapes) -> drag control points on the map -> SAVE -> <code>DeploymentService.updateControlPoints()</code> rebuilds the shape from the moved points and bumps its version.
Delete	Click DELETE -> <code>DeploymentController.delete()</code> -> the deployment and all its linked rows are removed.
App startup	<code>DataInitializer</code> seeds default settings, then <code>AppStartupRunner</code> scans the terrain folder and registers any map images it finds.

3. What gets stored (the database tables)

The database is built automatically on first run by the migration files. These are the tables and what each one holds:

Table	What it holds
deployment	One planned camp: its centre lat/lon, heading, name, and status.
deployment_parameters	The inputs used for it: frontage, depth, slope threshold, options.
deployment_geometry	The drawn shape, stored 3 ways (map format, text format, database format), plus a version and a valid/invalid flag.
control_point	The draggable corners of a blob shape (position + curve handles).
terrain_analysis	The terrain results: elevations, slopes, roughness, suitability, flat-or-not.
slope_analysis	Individual sampled ground points (height, slope, aspect) for one analysis.
terrain_tile	A record of each terrain file on disk and the area it covers.
threshold_config	Adjustable slope / roughness / suitability settings.
frontage_config	Adjustable frontage min / max / default / step.
depth_config	Adjustable depth min / max / default / step.

Note: A deployment is the 'parent'. Its parameters, geometry, terrain analysis, and control points all point back to it. Delete the deployment and the database removes all of them automatically (that is the 'ON DELETE CASCADE' rule in the migrations).

4. Backend files (the Java 'kitchen')

Location: backend/src/main/java/in/drdo/gis/ unless noted otherwise.

4.1 Start-up and configuration (config/)

File	What it does (plain English)
<code>GisDeploymentApplication.java</code>	The backend's 'start button'. Boots the whole Spring application.
<code>config/GisProperties.java</code>	Loads all the <code>gis.*</code> settings (terrain paths, geometry tuning numbers) so code can read them.
<code>config/WebConfig.java</code>	The clever trick that lets ONE app serve both the website (at /) and the API (at /api). Also handles page refreshes and dev-mode CORS.
<code>config/JpaConfig.java</code>	Turns on database features (repositories + transactions) and stamps 'system' as the author on saved rows.
<code>config/OpenApiConfig.java</code>	Names and labels the auto-generated API documentation (Swagger).
<code>config/DataInitializer.java</code>	On first start, inserts one default settings row if it is missing.
<code>config/AppStartupRunner.java</code>	On every start, scans the terrain folder and registers any background map images (GeoTIFFs) it finds.

4.2 The engines - the brain (engine/)

These do the real math. They are the most important files to understand.

File	What it does (plain English)
<code>engine/TerrainEngine.java</code>	THE decision-maker. Samples ground heights, measures slope and bumpiness, decides flat vs hilly, and scores suitability (0-100%).
<code>engine/DtedReader.java</code>	Reads the military elevation files (.dt1) and returns the ground height at any point. Keeps loaded tiles in memory so huge areas are fast.
<code>engine/GeometryEngine.java</code>	Chooses and builds the final shape: an oval for flat ground, a terrain-avoiding blob for hilly ground.
<code>engine/BezierEngine.java</code>	The actual 'pencil'. Draws smooth ovals and smooth curved (Bezier) polygons from a set of points.
<code>engine/SlopeEngine.java</code>	Standalone slope / aspect / roughness math helpers, kept separate so they are easy to test on their own.
<code>engine/GeoTiffReader.java</code>	Reads the background map image files (GeoTIFF): their size, where they sit on Earth, and pixel values.
<code>engine/ProjectionService.java</code>	Converts coordinates between formats (lat/lon and UTM) and measures metres-per-degree accurately.

4.3 The services - the managers (service/)

File	What it does (plain English)
<code>service/DeploymentService.java</code>	The head chef. Runs the full create / list / get / update / delete workflow and ties every engine and repository together.
<code>service/GeometryEditingService.java</code>	Handles the 'drag the control points and re-draw' edit, validating and repairing the new shape.
<code>service/TerrainAnalysisService.java</code>	Re-runs the terrain analysis for an existing deployment when asked.
<code>service/TileManagementService.java</code>	Finds and registers background map files, and safely serves them to the website (guards against path tricks).
<code>service/PolygonValidationService.java</code>	Checks a shape is geometrically valid (no self-crossing) and repairs it if not.
<code>service/ConfigService.java</code>	Reads and updates the adjustable settings (slope threshold, size limits) for the UI.
<code>service/CoordinateConversionService.java</code>	Wraps ProjectionService so the website can request coordinate conversions over the API.

4.4 The controllers - the doors (controller/)

All are reached under `/api/v1/...`

File	What it does (plain English)
<code>controller/DeploymentController.java</code>	Doors for creating, listing, viewing, editing, and deleting deployments.
<code>controller/TerrainController.java</code>	Door to fetch a deployment's terrain analysis.
<code>controller/TerrainRecomputeController.java</code>	Door to recompute terrain for a deployment.
<code>controller/GeometryEditingController.java</code>	Door for the geometry-edit endpoint.
<code>controller/TileController.java</code>	Doors to list map files and stream a map image (supports chunked downloads that OpenLayers needs).
<code>controller/CoordinateController.java</code>	Doors for coordinate conversions (lat/lon <-> UTM, etc.).
<code>controller/ValidationController.java</code>	Door to check whether a given shape (as WKT text) is valid.
<code>controller/ConfigController.java</code>	Doors to read settings and change the slope threshold.

4.5 The entities - the shape of stored data (entity/)

File	What it does (plain English)
<code>entity/Deployment.java</code>	The main record: one planned camp (location, heading, status) linked to its parts.
<code>entity/DeploymentParameters.java</code>	The inputs used: frontage, depth, slope threshold, heading, options.
<code>entity/DeploymentGeometry.java</code>	The drawn shape stored 3 ways, plus version and valid/invalid flag.

File	What it does (plain English)
<code>entity/ControlPoint.java</code>	One draggable corner of a blob shape (its position + two curve handles).
<code>entity/TerrainAnalysis.java</code>	The terrain results: elevations, slopes, roughness, suitability, flat-or-not.
<code>entity/SlopeAnalysis.java</code>	One sampled ground point (height, slope, aspect) belonging to a terrain analysis.
<code>entity/TerrainTile.java</code>	A record of one terrain file on disk (its type and the area it covers).
<code>entity/ThresholdConfig.java</code>	Adjustable slope / roughness / suitability settings row.
<code>entity/FrontageConfig.java</code>	Adjustable frontage min / max / default / step row.
<code>entity/DepthConfig.java</code>	Adjustable depth min / max / default / step row.

4.6 The repositories - fetch and save (repository/)

File	What it does (plain English)
<code>repository/DeploymentRepository.java</code>	Find / save deployments; includes map-based 'near this point' and 'inside this box' searches.
<code>repository/DeploymentGeometryRepository.java</code>	Find a deployment's shape; find shapes inside a map box.
<code>repository/ControlPointRepository.java</code>	Find and delete a deployment's control points.
<code>repository/TerrainAnalysisRepository.java</code>	Find a deployment's terrain analysis.
<code>repository/SlopeAnalysisRepository.java</code>	Find and delete the sampled points of an analysis.
<code>repository/TerrainTileRepository.java</code>	Find terrain files covering a point or a box.
<code>repository/DeploymentParametersRepository.java</code>	Find a deployment's input parameters.
<code>repository/ThresholdConfigRepository.java</code>	Find threshold settings by key.
<code>repository/FrontageConfigRepository.java</code>	Find frontage settings by key.
<code>repository/DepthConfigRepository.java</code>	Find depth settings by key.

4.7 The DTOs - the envelopes (dto/)

File	What it does (plain English)
<code>dto/DeploymentRequestDto.java</code>	Incoming 'create' data, and the validation rules (allowed lat/lon/size ranges) live here.
<code>dto/DeploymentResponseDto.java</code>	Outgoing full reply (camp + parameters + geometry + terrain).

File	What it does (plain English)
<code>dto/DeploymentParametersDto.java</code>	Outgoing parameters block.
<code>dto/DeploymentGeometryDto.java</code>	Outgoing geometry block (shape + type + validity).
<code>dto/TerrainAnalysisDto.java</code>	Outgoing terrain-results block.
<code>dto/ControlPointDto.java</code>	One control point inside requests and replies.
<code>dto/ControlPointUpdatedDto.java</code>	The list of control points sent when saving an edit.
<code>dto/GeoJsonFeatureDto.java</code>	A flexible GeoJSON 'Feature' holder (a helper, not on the main path).

4.8 Glue: mapper, errors, utilities

File	What it does (plain English)
<code>mapper/DeploymentMapper.java</code>	Auto-converts entities to DTOs (MapStruct). The main service mostly does this by hand instead.
<code>exception/GlobalExceptionHandler.java</code>	Catches errors anywhere and returns tidy JSON with the correct HTTP status code.
<code>exception/DeploymentNotFoundException.java</code>	'That deployment does not exist' error -> becomes HTTP 404.
<code>exception/GeometryException.java</code>	'Could not build or repair the shape' error -> becomes HTTP 422.
<code>exception/TerrainDataException.java</code>	'Terrain / map file problem' error -> becomes HTTP 422.
<code>util/GeoUtils.java</code>	Small geo math: distance, bearing, offset a point by metres, make a point.
<code>util/GeoJsonWriter.java</code>	Turns a shape into GeoJSON text that the map can draw.

4.9 Resources and build

File	What it does (plain English)
<code>resources/application.yml</code>	Master settings baked into the app: database login, terrain folder paths, geometry tuning numbers.
<code>resources/db/changelog/db.changelog-master.xml</code>	Database build instructions #1: turn on PostGIS and create all tables + indexes.
<code>resources/db/changelog/db.changelog-002.xml</code>	Database build instructions #2: create the frontage/depth config tables and seed defaults.
<code>resources/static/ (many .js/.css)</code>	The prebuilt website bundle packed into the app. Auto-generated - never edit by hand.
<code>pom.xml</code>	The Maven build recipe: lists every library used and how to build the single runnable file.

4.10 Tests (backend/src/test/...)

File	What it does (plain English)
<code>engine/TerrainEngineTest.java</code>	Checks that flat ground is classified as planar and scores well.
<code>engine/BezierEngineTest.java</code>	Checks ovals and curves are built correctly.
<code>engine/SlopeEngineTest.java</code>	Checks the slope / aspect math.
<code>engine/ProjectionServiceTest.java</code>	Checks lat/lon <-> UTM conversion.
<code>engine/DtedDiscoveryTest.java</code>	Checks terrain files are found by their own header, whatever the folder layout.
<code>util/GeoUtilsTest.java</code>	Checks the distance / bearing helpers.
<code>service/PolygonValidationServiceTest.java</code>	Checks that invalid shapes are detected and repaired.
<code>controller/DeploymentControllerIT.java</code>	End-to-end test that calls the real 'create deployment' API.

5. Frontend files (the Angular 'dining area')

Location: frontend/

5.1 Project setup / scaffolding

File	What it does (plain English)
<code>package.json</code>	Lists the website's libraries (Angular, OpenLayers) and the build commands.
<code>package-lock.json</code>	The exact locked versions of those libraries (auto-generated).
<code>angular.json</code>	Angular build settings: entry files, output folder, size budgets.
<code>tsconfig.json</code>	TypeScript compiler settings.
<code>src/index.html</code>	The single HTML page the whole app loads into.
<code>src/main.ts</code>	The website's 'start button': boots the Angular app.
<code>src/styles.scss</code>	Global styles for the whole site.
<code>src/environments/environment.ts</code>	Dev settings: API at localhost:8080, map centred on India.
<code>src/environments/environment.prod.ts</code>	Production settings: API at /api/v1 (same server as the UI).

5.2 App shell and shared helpers

File	What it does (plain English)
<code>app/app.module.ts</code>	Wires all website pieces together (declares components, imports modules, registers the error interceptor).
<code>app/app-routing.module.ts</code>	The routing table. Here it simply shows the map screen for every path.
<code>app/app.component.ts</code>	The tiny root component that hosts the routed page.
<code>app/shared/shared.module.ts</code>	Bundles shared UI helpers (the coordinate pipe) for reuse.
<code>app/shared/pipes/coordinate-format.pipe.ts</code>	Formats a decimal like 28.6 into '28 deg 36' 0.0" N'.

5.3 Core - data shapes, phone lines, shared state

File	What it does (plain English)
<code>app/core/models/deployment.model.ts</code>	TypeScript definitions mirroring the backend's data shapes.
<code>app/core/services/deployment-api.service.ts</code>	Main phone line to the backend (create / list / get / edit / delete).
<code>app/core/services/tile-api.service.ts</code>	Phone line to list background maps and build a map image URL.
<code>app/core/services/config-api.service.ts</code>	Phone line to read and update settings.

File	What it does (plain English)
<code>app/core/services/coordinate.service.ts</code>	Phone line for coordinate conversions.
<code>app/core/services/map-state.service.ts</code>	A small in-app memory of 'what is selected right now', shared between components.
<code>app/core/interceptors/api-error.interceptor.ts</code>	Catches failed API calls and turns them into clean error messages.

5.4 Map feature - the main screen

File	What it does (plain English)
<code>app/features/map/components/map-view.component.ts</code>	The main screen's brain: handles map clicks, form submit, select/delete, and edit mode.
<code>app/features/map/components/map-view.component.html</code>	The main screen's layout: toolbar, map, left list, right panels, hints.
<code>app/features/map/components/map-view.component.scss</code>	The main screen's styling.
<code>app/features/map/services/ol-map.service.ts</code>	All the OpenLayers map work: base map, drawing shapes, dragging points, zooming.

5.5 Deployment feature - form, list, editor

File	What it does (plain English)
<code>app/features/deployment/components/deployment-form.component.ts</code>	The 'new deployment' form logic and input validation.
<code>deployment-form.component.html / .scss</code>	The form's layout and styling.
<code>app/features/deployment/components/deployment-list.component.ts</code>	The left-side list of saved deployments.
<code>deployment-list.component.html / .scss</code>	The list's layout and styling.
<code>app/features/deployment/components/geometry-editor.component.ts</code>	A numeric editor to type exact control-point coordinates.
<code>geometry-editor.component.html / .scss</code>	The editor's layout and styling.
<code>app/features/deployment/services/geometry-editing.service.ts</code>	Phone line to the geometry-edit API endpoint.

5.6 Terrain feature - the results panel

File	What it does (plain English)
<code>app/features/terrain/components/terrain-panel.component.ts</code>	Shows suitability %, flat/hilly, slope, elevation; colour-codes it green/amber/red.
<code>terrain-panel.component.html / .scss</code>	The panel's layout and styling.
<code>app/features/terrain/services/terrain-api.service.ts</code>	Phone line to fetch or recompute terrain for a deployment.

6. Data, deployment, and project files

6.1 Terrain data (data/)

File	What it does (plain English)
<code>data/terrain/dted/nXX/eYYY.dt1</code>	Sample elevation files (ground heights) covering the Delhi / Haryana area. Replace with DRDO's own for other regions.
<code>data/terrain/geotiff/india_basemap.tif</code>	The India background-map image shown under everything. Must be in Web-Mercator (EPSG:3857).

6.2 Ready-to-run app (deploy/manual/)

File	What it does (plain English)
<code>deploy/manual/backend.jar</code>	The whole prebuilt app in one runnable file (about 82 MB): API + website together.
<code>deploy/manual/run.sh</code>	Linux 'ON' button. Starts the app and points it at the data folder.
<code>deploy/manual/run.bat</code>	Windows 'ON' button (does the same thing).
<code>deploy/manual/application.properties</code>	Editable settings read at every start (database login, port, terrain paths) - no rebuild needed.

6.3 Project root

File	What it does (plain English)
<code>README.md</code>	The complete offline setup and troubleshooting manual - every error and its fix.
<code>.gitignore</code>	Tells Git which files never to save (build outputs, downloaded libraries).

7. Glossary - terms in plain words

File	What it does (plain English)
DTED	A standard military file format that stores ground heights on a grid. The app reads .dt1 files to know how high/steep the land is.
GeoTIFF	An image file (like a photo of the ground) that also knows where on Earth it sits, so it lines up correctly as a map background.
Slope	How steep the ground is, in degrees. 0 = perfectly flat.
Roughness	How bumpy/irregular the ground is, ignoring overall tilt. A smooth slanted hill is low-roughness; broken ground is high.
Planar	The app's word for 'flat enough to treat as flat' -> it draws an oval.
Bezier	A way to draw smooth curves from a few control points. Used for the terrain-avoiding blob shapes.
Control point	A draggable corner of a blob shape. Moving it reshapes the geometry.
Ellipse	An oval. The shape drawn on flat ground.
Frontage / Depth	The width and length of the deployment, in metres.
Heading	The direction the deployment faces, in degrees (0 = North).
Suitability	A 0-100% score for how good the ground is: mostly from gentle slope, partly from smoothness.
GeoJSON	A text format for map shapes that the website's map library can draw directly.
WKT	'Well-Known Text' - another text way to write a shape, e.g. POLYGON((...)).
PostGIS	A map/location add-on for the PostgreSQL database, so it can store and search shapes.
Spring Boot	The Java framework the backend is built on. It wires everything together and runs the web server.
Angular / OpenLayers	Angular builds the website; OpenLayers draws the interactive map inside it.
DTO	A plain 'envelope' object used only to carry data between the website and the backend.
Transaction	A group of database steps that all succeed together or all get undone - so data never ends up half-saved.

One-line summary to end on

Note: Click a spot -> DtedReader reads the ground height -> TerrainEngine decides flat or hilly -> GeometryEngine + BezierEngine draw the shape -> it is saved and shown on the map. Every other file is a helper around that one sentence.